Unit-4 – Question 1

Object-Oriented Principles in Python

Question 1: Explain object-oriented principles with examples.

① What is Object-Oriented Programming (OOP)?

- OOP is a programming paradigm based on the concept of "objects".
- It organizes software as a collection of interacting objects.
- It is built on four main principles: Encapsulation, Abstraction, Inheritance and Polymorphism.

② Class and Object

- Class is a blueprint or template that defines attributes (data) and methods (functions).
- Object is an instance of a class.
- Through objects we access the members of the class.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name, self.age)

s = Student("Ravi", 20)
s.display()      # Ravi 20
```

③ Encapsulation (Data Hiding)

- Bundling data and methods together in a class.
- Restrict direct access to some data using private variables.
- In Python, prefixing __ (double underscore) makes a variable name-mangled (treated as private). Getter / Setter methods provide controlled access.

```
class Student:
    def __init__(self, name, marks):
        self.__marks = marks   # private
        self.name = name
    def get_marks(self):
        return self.__marks
    def set_marks(self, m):
        if 0 <= m <= 100:
            self.__marks = m
        else:
            print("Invalid marks")

s = Student("Neha", 85)
print(s.get_marks())   # 85
s.set_marks(90)
```

④ Abstraction

- Hiding internal implementation details and showing only the essential features to the user.
- Achieved using abstract classes and methods (from abc module) or by providing a clean interface.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r

c = Circle(5)
print(c.area())  # 78.5
```

⑤ Inheritance

- Creating a new class (derived / child) from an existing class (base / parent).
- The child class inherits attributes and methods of the parent class.
- Promotes code reusability and establishes "is-a" relationship.

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):  # Dog is an Animal
    def speak(self):
        print("Dog barks")
d = Dog()
d.speak()      # Dog barks
```

⑥ Polymorphism

- Means "many forms". Same interface, different behavior.
- Achieved by method overriding or operator overloading.
- Python supports operator overloading.

```
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag
    def __add__(self, other):     # operator overloading
        return Complex(self.real + other.real,
                       self.imag + other.imag)

c1 = Complex(2, 3)
c2 = Complex(4, 5)
c3 = c1 + c2
print(c3.real, c3.imag)  # 6 8
```

⑦ Advantages of OOP

- Modularity : Code is divided into classes and objects.
- Reusability : Inheritance and classes allow code reuse.
- Maintainability : Easy to update and maintain large programs.

⑧ Conclusion

- OOP principles (Encapsulation, Abstraction, Inheritance, Polymorphism) help in designing well-structured, reusable, and scalable programs. Python supports these principles effectively, making it an ideal language for object-oriented development.

Unit-4 - Question 2

Class and Object in Python

Question 2: Define class and object? How to create the class and object? How to access the members of class using the object reference?

① Definition:

- **Class :** A class is a blueprint or template for creating objects. It defines a set of attributes (data) and methods (functions) that the objects created from the class will have.
- **Object :** An object is an instance of a class. It is a real-world entity that has state (data) and behavior (methods) as defined by the class.

② Creating a class (Syntax):

```
class ClassName:
    def __init__(self, parameters):
        # initialization of data members

    def method_name(self, parameters):
        # method body
```

→ self refers to the current object and is used to access its members.

③ Creating objects (Syntax):

```
obj_ref = ClassName(arguments)
```

→ obj_ref is the object reference (variable) that points to the created object.

④ Accessing members using object reference:

- Accessing data members (attributes): `obj_ref.attribute_name`
- Calling methods (member functions): `obj_ref.method_name(arguments)`
- Updating data members: `obj_ref.attribute_name = new_value`

⑤ Example:

```
class Student:
    def __init__(self, name, roll):
        self.name = name        # data members (attributes)
        self.roll = roll

    def display(self):          # method
        print(f"Name: {self.name}, Roll: {self.roll}")

# Creating objects
s1 = Student("Alice", 101)
s2 = Student("Bob", 102)

# Accessing data members
print("s1.name =", s1.name)
print("s1.roll =", s1.roll)

# Updating a data member
s2.roll = 105

# Calling method
s1.display()
s2.display()
```

Expected Output :

```
s1.name = Alice
s1.roll = 101
Name: Alice, Roll: 101
Name: Bob, Roll: 105
```

⑥ Why classes and objects are useful?

- Promote code reusability by creating blueprints for similar entities.
- Help in organizing code in a modular and manageable way.
- Support encapsulation - bundling data and methods together.
- Facilitate easy maintenance and extension of programs.

Summary: A class defines what an object will have (attributes and methods), and an object is a specific instance of that class. We use the object reference to access and manipulate the members of the class.

Unit-4 - Question 3

Constructor, Method, Destructor

Question 3: Write a short notes on following terms:
i) Constructor ii) Method iii) Destructor

① Constructor

- **Definition :** A constructor is a special member function that is automatically called when an object is created.
- **Purpose :** Used to initialize the object and set initial values to instance variables.
- **Important Points :**
  - Name of constructor is always `__init__()`.
  - It is automatically invoked when an object of the class is created.
  - It can take arguments (parameters).
  - It does not return any value.
  - Every class can have only one `__init__()` method.

Syntax Idea

```
def __init__(self, [parameters]):
    # initialization code
```

Example

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p = Person("Alice", 20)   # __init__ called automatically
```

② Method

- **Definition :** A method is a function defined inside a class and is used to perform operations on objects.
- **Purpose :** Defines the behavior of an object. It operates on the data (instance variables) of the object.
- **Important Points :**
  - The first parameter is always self, which refers to the current object.
  - Methods can access and modify instance variables.
  - Can have parameters and return values.
  - Called using the object reference with dot (.) operator.

Syntax Idea

```
def method_name(self, [parameters]):
    # method body
```

Example

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello, I am", self.name)

p = Person("Alice")
p.greet()     # method call
```

③ Destructor

- **Definition :** A destructor is a special member function that is called when an object is destroyed or garbage collected.
- **Purpose :** Used to clean up resources or perform any final operations before the object is removed from memory.
- **Important Points :**
  - Name of destructor is always `__del__()`.
  - It is not called explicitly; Python calls it automatically.
  - It is called when the object is about to be destroyed (e.g., reference count becomes zero or by garbage collection).
  - Its call time is not guaranteed, it depends on the interpreter.

Syntax Idea

```
def __del__(self):
    # cleanup code
```

Example

```
class FileHandle:
    def __init__(self, filename):
        self.f = open(filename, 'r')

    def __del__(self):
        self.f.close()
        print("File closed.")
```

④ Combined Example Using Constructor, Method and Destructor

```
class Demo:
    def __init__(self, value):              # Constructor
        self.value = value
        print("Object created with value =", self.value)
    def show(self):                         # Method
        print("Current value =", self.value)
    def __del__(self):                      # Destructor
        print("Object with value", self.value, "is destroyed.")
```

Usage

```
d = Demo(10)     # __init__ called
d.show()         # method called
del d            # __del__ may be called
                 # automatically
```

⑤ Summary

- Constructor (`__init__`): Initializes the object; called automatically at creation.
- Method: Defines behavior; called using object reference.
- Destructor (`__del__`): Cleans up; called automatically when object is destroyed / garbage collected.

Unit-4 - Question 4

Types of Methods in Classes

Question 4: Explain different types of methods used in classes? How to call those methods?

① Instance Method

- **Definition :** Methods that are bound to an object (instance) of the class. They operate on instance data using 'self'.
- **Syntax :** `def method_name(self, ...):`
- **Uses :** Access and modify instance variables. Perform actions related to a specific object.
- **How to call :** Create an object and call using dot operator. `obj.method_name(...)`

② Class Method

- **Definition :** Methods that are bound to the class and take 'cls' as the first parameter. They can access and modify class variables.
- **Syntax :**
```
@classmethod
def method_name(cls, ...):
```
- **Uses :** Work with class-level data. Provide alternative constructors or factory methods.
- **How to call :** Call using the class name. `ClassName.method_name(...)`

③ Static Method

- **Definition :** Methods that do not operate on instance or class data. They do not take 'self' or 'cls'. They act like regular functions inside a class.
- **Syntax :**
```
@staticmethod
def method_name(...):
```
- **Uses :** Utility/helper functions related to the class. Operations that do not need class or instance data.
- **How to call:** Call using the class name (preferred) or object. `ClassName.method_name(...)` or `obj.method_name(...)`

Example:

```
class Demo:
    class_name = "Python"          # class variable
    def __init__(self, value):
        self.value = value         # instance variable
    def show(self):                # instance method
        print(f"Instance value: {self.value}")
    @classmethod
    def class_method(cls):         # class method
        print(f"Class name: {cls.class_name}")
    @staticmethod
    def static_method():           # static method
        print("Hello from static method")
```

Creating object and calling methods:

```
obj = Demo(10)          # create object
obj.show()              # call instance method
Demo.class_method()     # call class method
Demo.static_method()    # call static method
```

Output:

```
Instance value: 10
Class name: Python
Hello from static method
```

Summary / Comparison

Type of Method	First Parameter	Uses	Can Access	How to Call	Uses self?	Uses cls?	Uses Neither?
Instance Method	self	Works with instance data	Instance vars, Class vars	obj.method()	Yes	No	No
Class Method	cls	Works with class data	Class vars, Other class data	ClassName.method()	No	Yes	No
Static Method	None	Utility functions (no data needed)	Nothing directly (not instance/class)	ClassName.method() or obj.method()	No	No	Yes

Unit-4 – Question 5

Variables in Classes

Question 5: Define variable? Explain different types of variables in classes? How to access those variables?

① Definition:

A variable is a named memory location used to store data that can change during program execution.

② Types of variables in classes:

- **Instance variable (Object / Non-static variable):**
  - Declared inside methods (usually `__init__`) using self.
  - Each object has its own separate copy.
  - **Access:** using object reference → obj.varname
- **Class variable (Static variable):**
  - Declared inside the class but outside any method.
  - Shared by all objects of the class.
  - **Access:** using ClassName.varname or obj.varname
- **Local variable (in methods):**
  - Declared inside a method.
  - Exists only within that method's scope.
  - **Access:** can be used only inside the method.

③ Example:

```
class Example:
    class_var = 100          # class (static) variable

    def __init__(self, x):
        self.x = x           # instance variable

    def show(self):
        y = self.x * 2       # local variable
        print("Inside method, y =", y)
        return y
```

④ Accessing variables:

- Instance variable (x): access via object → obj.x
- Class variable (class_var): access via ClassName.class_var or obj.class_var
- Local variable (y): accessible only inside the method show()

⑤ Demonstration:

```
e1 = Example(10)
e2 = Example(20)
print("e1.x =", e1.x)                          # 10
print("e2.x =", e2.x)                          # 20
print("Example.class_var =", Example.class_var) # 100
print("e1.class_var =", e1.class_var)          # 100
print(e1.show())                               # Inside method, y = 20 → 20
print(e2.show())                               # Inside method, y = 40 → 40
```

Output / Observations:

- Each object has its own instance variable (x).
- Class variable is shared by all objects.
- Local variable (y) exists only during method call and is not accessible outside.

⑥ Key Notes:

- If an instance variable has the same name as a class variable, the instance variable hides the class variable for that object.
- Use instance variables for object-specific data.
- Use class variables for data shared by all objects (constants, counters, etc.).
- Local variables help to keep temporary data within the method (good for encapsulation).

Unit-4 - Question 6

Inheritance in Python

Question 6: Define Inheritance? Explain different types of inheritances supported by python.

① Definition: Inheritance is a mechanism in OOP that allows a new class (child/derived class) to acquire the properties and behaviors (attributes and methods) of an existing class (parent/base class). It promotes code reusability and establishes an "is-a" relationship.

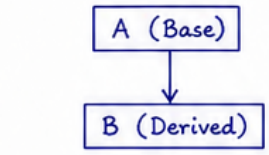
② Terms:

- Parent / Base class: The class whose members are inherited.
- Child / Derived class: The class that inherits members from the parent class.

③ Types of Inheritance in Python:

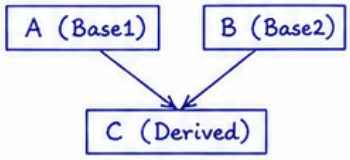
i. **Single Inheritance:**
A child class inherits from a single parent class.

```
class A:
    pass
class B(A):
    pass
```



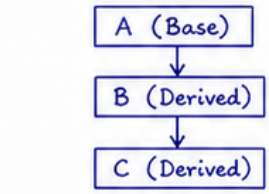
ii. **Multiple Inheritance:**
A child class inherits from more than one parent class.

```
class A:
    pass
class C(A, B):
    pass
class B:
    pass
```



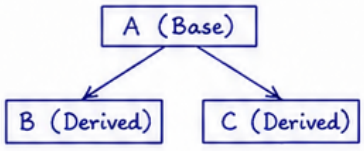
iii. **Multilevel Inheritance:**
A derived class becomes the base class for another derived class.

```
class A:
    pass
class B(A):
    pass
class C(B):
    pass
```



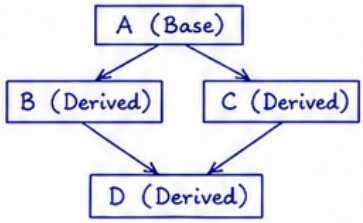
iv. **Hierarchical Inheritance:**
Multiple child classes inherit from the same parent class.

```
class A:
    pass
class B(A):
    pass
class C(A):
    pass
```



v. **Hybrid Inheritance:**
A combination of two or more types of inheritance.

```
class A:
    pass
class B(A):
    pass
class C(A):
    pass
class D(B, C):
    pass
```



④ Advantages of Inheritance:

- Code reusability: Inherited members can be reused, saving development time.
- Reduces redundancy: Avoids writing the same code again and again.
- Extensibility: New features can be added by creating new derived classes.
- Better maintainability: Changes in base class reflect in all derived classes.
- Polymorphism support: Enables method overriding for flexible designs.

⑤ Summary:

Inheritance is a powerful OOP feature that allows a class to inherit attributes and methods from another class. Python supports single, multiple, multilevel, hierarchical and hybrid inheritance, which help in building reusable, extensible and maintainable programs.

Unit-4 – Question 7 and 8

Overloading, Overriding and super()

Question 7: Explain following terms.

① **Operator Overloading**: Allows user-defined classes to give special meaning to built-in operators (+, -, *, etc.). Implemented using magic methods (dunder methods).

Example: Overload + to add two Complex numbers.

```
class Complex:
    def __init__(self, r=0, i=0):
        self.r = r; self.i = i
    def __add__(self, other):      # self + other
        return Complex(self.r + other.r, self.i + other.i)
    def __str__(self):
        sign = '+' if self.i >= 0 else ''
        return f"{self.r} {sign} {self.i}i"

c1 = Complex(2, 3)
c2 = Complex(4, -1)
c3 = c1 + c2       # uses __add__
print(c3)          #  6 + 2i
```

② **Method Overloading and Overriding**:

- Method overloading in Python is limited. Achieved using default arguments or variable arguments (*args, **kwargs).

  Example (overloading using default/variable arguments):

```
class Calc:
    def add(self, a, b=0, c=0):   # one method, many uses
        return a + b + c

obj = Calc()
print(obj.add(5))          # 5
print(obj.add(5, 10))      # 15
print(obj.add(5, 10, 15)) # 30
```

- Method overriding: Child class provides its own implementation of a method already defined in the parent class (same name and parameters).

  Example (overriding):

```
class Animal:
    def sound(self):      return "Some sound"
class Dog(Animal):
    def sound(self):      return "Bark"

a = Animal(); d = Dog()
print(d.sound())      #  Some sound
print(d.sound())      #  Bark
```

③ **Constructor Overloading and Overriding**:

- Constructor overloading: Python does not support multiple constructors with same name, but we achieve it using default/variable arguments.

  Example (overloading using default/variable arguments):

```
class Person:
    def __init__(self, name, age=0, city="Unknown"):
        self.name = name; self.age = age; self.city = city
p1 = Person("Ravi")                  # only name
p2 = Person("Ravi", 21)              # name and age
p3 = Person("Ravi", 21, "Pune")      # all three
```

- Constructor overriding (in inheritance): Child class defines its own __init__() and usually calls parent constructor using super().

  Example (overriding):

```
class Student(Person):
    def __init__(self, name, age, city, rollno):
        super().__init__(name, age, city)  # call parent constructor
        self.rollno = rollno
```

Question 8: Explain super() function with example.

super() returns a proxy object that allows a child class to call methods or constructors of its parent class.

Uses:

1. To call parent class constructor.
2. To call parent class methods.

Example:

```
class Person:
    def __init__(self, name):
        self.name = name
    def show(self):
        print(f"Person: {self.name}")

class Student(Person):
    def __init__(self, name, rollno):
        super().__init__(name)  # 1. call parent constructor
        self.rollno = rollno

    def show(self):
        super().show()       # 2. call parent method
        print(f"Student Roll No: {self.rollno}")

s = Student("Ravi", 101)
s.show()

# Output:
# Person: Ravi
# Student Roll No: 101
```

Note:

- super() should be used in new-style classes (Python 3).
- It supports multiple inheritance and method resolution order (MRO).

Quick Comparison

Term	What it is	When used	Example
Operator Overloading	Giving special meaning to operators.	For user-defined objects with operators.	__add__, __sub__, __eq__, etc.
Method Overloading	Same method name, different ways (via defaults/ *args/**kwargs).	Same class.	add(a), add(a,b), add(a,b,c)
Method Overriding	Child class provides its own method (same signature).	Inheritance (parent-child).	sound() in Animal/Dog
Constructor Overloading	Different ways to create object (via defaults/ *args/**kwargs).	Same class.	__init__(a) __init__(a,b) __init__(a,b,c)
Constructor Overriding	Child class defines its own __init__ (usually calls parent via super()).	Inheritance (child constructor replaces parent).	super().__init__() in child __init__

Summary: Overloading → many forms in same class (operators/methods/constructors).
Overriding → child class redefines behavior of parent (methods/constructors).
super() → bridge to access parent constructor and methods.

Unit-4 – Question 9

File Handling Functions in Python

Question 9: Explain the file handling functions with examples.

File handling in Python:

File handling allows us to store, read and write data permanently in files.

General Steps:

1. Open the file.
2. Perform the required operation (read / write / update).
3. Close the file.

① **open()**: Opens a file and returns a file object.
Syntax: open(filename, mode)
Common modes: 'r' read, 'w' write, 'a' append, 'r+' read & write.

```
f = open('data.txt', 'r')   # open file for reading
```

② **read()**: Reads the entire content of the file as a single string.

```
data = f.read()   # read entire file
```

③ **readline()**: Reads one line at a time.

```
line = f.readline()   # read one line
```

④ **readlines()**: Reads all lines and returns them as a list of strings.

```
lines = f.readlines()   # read all lines as list
```

⑤ **write()**: Writes a string to the file.

```
f.write('Hello Python\n')   # write string
```

⑥ **writelines()**: Writes a sequence of strings (line by line) to the file.

```
f.writelines(['Line 1\n', 'Line 2\n'])   # write multiple lines
```

⑦ **seek(offset, whence)**: Changes the current position in the file.
- offset : number of characters
- whence : 0 = beginning, 1 = current position, 2 = end of file

```
f.seek(0)   # move to beginning
```

⑧ **tell()**: Returns the current position of the file pointer.

```
pos = f.tell()   # get current position
```

⑨ **close()**: Closes the file and releases the resources.

```
f.close()   # close file
```

⑩ **with open()**: Best Practice
Automatically closes the file even if an error occurs.

```
with open('data.txt','r') as f:
    data = f.read()
    print(data)
# file closed automatically
```

Example 1: Reading a file

```
f = open('sample.txt', 'r')
text = f.read()
print(text)
f.close()
```

Read entire content of sample.txt and print it

Example 2: Writing to a file

```
f = open('output.txt', 'w')
f.write('Hello World!\n')
f.write('Python is fun.\n')
f.close()
```

Write two lines to output.txt

Summary:

- Use open() to open a file in the required mode.
- Use read(), readline(), readlines() to read data.
- Use write(), writelines() to write data.
- Use seek() and tell() to move and check file position.
- Always close the file using close() or preferably use with open() statement.

Unit-4 – Questions 10 and 11

File Opening Modes, Log Files and Config Files

(10) Question 10: Explain file opening modes with examples.

- The open() function uses a mode string to control how a file is opened.

Mode	Meaning	Example
• 'r'	Read: open for reading (file must exist).	open('data.txt', 'r')
• 'w'	Write: open for writing (creates file or truncates if exists).	open('data.txt', 'w')
• 'a'	Append: open for appending (creates file if not exists).	open('data.txt', 'a')
• 'x'	Exclusive create: create file, fail if file exists.	open('new.txt', 'x')
• 'b'	Binary mode: data read/written in binary form.	open('img.dat', 'rb')
• 't'	Text mode (default): data read/written as text.	open('data.txt', 'rt')
• 'r+'	Read and write: open for both, file must exist.	open('data.txt', 'r+')
• 'w+'	Write and read: truncate/create, then read & write.	open('data.txt', 'w+')
• 'a+'	Append and read: create if not exists, read and append.	open('data.txt', 'a+')

Note: 1) Add 'b' for binary or 't' for text as needed.
2) Advanced updates (read & write) require pointer handling with f.seek().

(11) Question 11: Explain log files and config files with examples.

① Log files: Log files store runtime events, errors, warnings and information.

- Why used? For debugging, auditing, monitoring, and troubleshooting.
- Python logging module example:

```
import logging
logging.basicConfig(filename='app.log', level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logging.info('Application started')
logging.warning('Low disk space')
try:
    10/0
except ZeroDivisionError as e:
    logging.error(f'Error occurred: {e}')
```

Example (app.log):

```
2025-05-25 10:00:01 - INFO - Application started
2025-05-25 10:00:05 - WARNING - Low disk space
2025-05-25 10:00:06 - ERROR - Error occurred:
  division by zero
```

② Config files: Config files store settings and parameters used by an application.

- Why used? Separates configuration from code, easy to modify without changing program.
- Python configparser example (config.ini):

config.ini (file format)

```
[Database]
host = localhost
port = 3306
user = admin
password = secret

[Paths]
log_file = app.log
data_dir = /data/files
```

```
import configparser
config = configparser.ConfigParser()
config.read('config.ini')

host = config['Database']['host']    # localhost
port = config['Database']['port']    # 3306

log_file = config['Paths']['log_file'] # app.log

print(host, port, log_file)     # localhost 3306 app.log
```

③ Applications:

i. Log files: Web servers, applications, databases, security monitoring.
ii. Config files: Database credentials, API keys, file paths, app preferences.

④ Conclusion / Summary:

- File opening modes control how files are accessed.
- Log files help record events for debugging and monitoring.
- Config files keep settings separate and easy to manage.

Unit-4 - Question 12

Important Programs

Question 12: Programs.

① Student class with attributes and display method

```
class Student:
    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.marks = marks
    def display(self):
        print(f"Roll:{self.roll} Name:{self.name} Marks:{self.marks}")
s = Student(101, "Amit", 88)
s.display()
# Output: Roll:101 Name:Amit Marks:88
```

② Add two complex numbers using class and objects

```
class Complex:
    def __init__(self, r=0, i=0):
        self.r, self.i = r, i
    def __add__(self, other):
        return Complex(self.r + other.r, self.i + other.i)
    def __str__(self):
        sign = '+' if self.i >= 0 else ''
        return f"{self.r}{sign}{self.i}j"
c1 = Complex(2, 3); c2 = Complex(4, -1)
c3 = c1 + c2
print("Sum =", c3)    # Output: Sum = 6+2j
```

③ Bank Account class: deposit, withdraw, display

```
class Account:
    def __init__(self, accno, bal=0):
        self.accno, self.bal = accno, bal
    def deposit(self, amt):
        self.bal += amt
    def withdraw(self, amt):
        if self.bal >= amt: self.bal -= amt
        else: print("Insufficient Balance")
    def display(self):
        print(f"Acc:{self.accno} Balance:{self.bal}")
a = Account(1001, 500)
a.deposit(300); a.withdraw(200); a.display()
# Output: Acc:1001 Balance:600
```

④ Rectangle class: find area and perimeter

```
class Rectangle:
    def __init__(self, l, b): self.l, self.b = l, b
    def area(self): return self.l * self.b
    def perimeter(self): return 2*(self.l + self.b)
r = Rectangle(10, 5)
print("Area =", r.area())        # 50
print("Perimeter =", r.perimeter())  # 30
```

⑤ Single inheritance: Employee and Manager

```
class Employee:
    def __init__(self, name, sal): self.name, self.sal = name, sal
    def display(self): print(f"Employee: {self.name}, {self.sal}")
class Manager(Employee):
    def __init__(self, name, sal, dept):
        super().__init__(name, sal); self.dept = dept
    def display(self):
        print(f"Manager: {self.name}, {self.sal}, Dept:{self.dept}")
m = Manager("Ravi", 50000, "IT")
m.display()    # Manager: Ravi, 50000, Dept:IT
```

⑥ Multiple inheritance: Father, Mother, Child

```
class Father:
    def f(self): print("From Father")
class Mother:
    def m(self): print("From Mother")
class Child(Father, Mother):
    def c(self): print("From Child")
ch = Child(); ch.f(); ch.m(); ch.c()
# Output:
# From Father
# From Mother
# From Child
```

⑦ Method overriding example: Shape and Circle

```
class Shape:
    def area(self): return 0
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r * self.r
s = Shape(); c = Circle(5)
print("Shape area =", s.area())   # 0
print("Circle area =", c.area())  # 78.5
```

⑧ Operator overloading: add two objects

```
class Vector:
    def __init__(self, i, j): self.i, self.j = i, j
    def __add__(self, other):
        return Vector(self.i + other.i, self.j + other.j)
    def __str__(self): return f"({self.i}, {self.j})"
v1 = Vector(3, 4); v2 = Vector(1, 2)
v3 = v1 + v2
print("Result =", v3)     # Result = (4, 6)
```

⑨ File handling: read and display file contents

```
# file: data.txt
# (contains: Hello World)
f = open('data.txt', 'r')
content = f.read()
print(content)
f.close()
# Output:
# Hello World
```

⑩ File handling: write data into a file

```
f = open('out.txt', 'w')
f.write("Python File Handling\n")
f.write("This is written to a file.")
f.close()
# Output: Data written to out.txt
```

⑪ File handling: append data to a file

```
f = open('out.txt', 'a')
f.write("\nAppended line.")
f.close()
# Output: Additional line appended to out.txt
```

⑫ Count lines / words / characters in a file

```
f = open('data.txt', 'r')
text = f.read()
lines = text.splitlines()
num_lines = len(lines)
words = text.split()
num_words = len(words)
num_chars = len(text)
f.close()
print("Lines:", num_lines)
print("Words:", num_words)
print("Characters:", num_chars)
# Sample Output (for 'Hello World')
# Lines: 1
# Words: 2
# Characters: 11
```